

google-calender-connect-tool

Lightweight Python helper (`calender.calender`) for real Google Calendar CRUD operations. It wraps the official Google Calendar API client, handles OAuth tokens per user, and exposes simple methods (`read`, `write`, `modify`, `delete`) so you can manage reservations in a shared calendar with owner metadata.

Requirements

- Python 3.10+ (Google 建議 3.11 以上 · 3.10 將於 2026-10-04 停止支援)
- Dependencies: install via `python -m pip install -r requirements.txt`
 - `google-api-python-client`
 - `google-auth-httpplib2`
 - `google-auth-oauthlib`

OAuth Setup

1. Create a Google Cloud project and enable **Google Calendar API**.
2. Create OAuth client credentials (Installed App) and download `credentials.json` into `calender/`.
3. Share the target calendar (e.g., `martincalendartool@gmail.com`) with every Google account that needs write access (Permission: “可變更行程”).
4. Each account runs the OAuth flow once; token files are stored per user.

calender Object Highlights

- `from_oauth_credentials(credentials_file, token_label=..., calendar_id=...)`: runs the OAuth flow, caches tokens (stored as `token-<label>.json` next to the credentials file), verifies calendar access, and remembers the authenticated email.
- `write(summary, start, end, description=None, location=None)`: creates events and automatically tags `extendedProperties.private.owner` with the authenticated account so shared calendars can show who created the event.
- `read(time_min=None, time_max=None, max_results=250)`: lists events via Google Calendar API; includes caching fallback if needed.
- `modify(event_id, **changes)` / `delete(event_id)`: patch or remove events.
- Additional wrappers exist for the full `events.*` REST surface (`list_events`, `get_event`, `move_event`, etc.).

Demo Script

`demo.py` shows how to:

1. Instantiate a client bound to a shared calendar and specific user token:

```
from demo import build_client

client = build_client(token_label="martin0968872985",
calendar_id="martincalendartool@gmail.com")
```

2. Read upcoming events (only future ones) and dump them to `upcoming_events.json`.
3. Insert a persistent event (e.g., 2026/01/09 12:00–13:57 at 國立臺灣大學) which remains on the shared calendar and records the owner email.

Run the demo after sharing the calendar and installing dependencies:

```
python demo.py
```

The first run per user launches a browser for OAuth consent and creates `calender/token-<label>.json`. Subsequent calls reuse that token automatically. To switch users programmatically, call `build_client(token_label="another-user")` and ensure the calendar is shared with that account.

Test Suite

Real API integration tests live in `test/test.py`. They share the same OAuth credentials and hit Google Calendar directly (read, write, modify, delete). By default they target `martincalendartool@gmail.com` with a token label of `integration-tests`. Run them with:

```
python test/test.py
```

Make sure:

- `calender/credentials.json` exists and represents an OAuth Installed App client.
- The shared calendar grants “可變更行程”權限 to the Google account you’ll authorize during the first test run.
- Each account uses its own token label (set in `CalendarTestCase.TOKEN_LABEL` if needed) to keep credentials separate.

Service Account Option

For backend-only workflows, use `calender.from_service_account_file(credentials_file, calendar_id=...)`. Make sure the service account email has access to the target calendar or use domain-wide delegation.

Notes

- The helper normalizes datetimes to ISO-8601 UTC so naive vs aware `datetime` comparisons will not break.
- Owner metadata is stored under `event["extendedProperties"]["private"]["owner"]` to keep track of which Google account created/updated an event.
- Keep your `credentials.json` and token files secure; they grant access to the linked calendars.